Terrier Team | University of Glasgow

Discriminative Models & Learning to Rank

Information Retrieval H/M

Craig Macdonald

2026

1

Generative vs. Discriminative

Terrier Team | University of Glasgow

Most of the retrieval models presented thus far in this course (e.g. BIM, BM25, LM) fall into the category of *generative models*

- A generative model assumes that documents were "generated" (produced) from some underlying model (in the case of language modelling, a multinomial distribution) and uses training data to estimate the parameters of the model
  - Probability of belonging to a class (i.e. the relevant documents for a query) is then estimated using Bayes' Rule and the document model

[We don't mean generative ala autoregression text generation in LLMs]

2

2

Generative vs. Discriminative

Terrier Team | University of Glasgow

- A *discriminative* model estimates the probability of belonging to a class directly from the observed features of the document based on the training data
- Generative models perform well with low numbers of training examples
- Discriminative models usually perform better given enough training data
  - Can also easily incorporate many features

3

3

Discriminative Models

Terrier Team | University of Glasgow

Discriminative models typically rely on machine learning

There has been considerable interaction between these fields

- 60s: Rocchio algorithm (c.f. Lecture 6) is a simple learning approach, and later in 80s & 90s other learned ranking algorithms based on user feedback were proposed
- 2000s: text classification

All were limited by low amounts of training data

Web query logs have generated a wave of research

- From 2008 e.g., "**Learning to Rank**"

Neural nets have given rise to more efforts in text representation:

- C.f. Word2vec, Glove, BERT

4

4

Motivations for Learning

Terrier Team | University of Glasgow

How to choose term weighting models?

- Term weighting models have different **assumptions** about how relevant documents should be retrieved (c.f. Lecture 7)

Also:

- **Field-based models**: term occurrences in different fields matter differently
- **Proximity-models**: close co-occurrences matter more
- **Priors**: documents with particular lengths or URL/inlink distributions matter more
- ***Query Features:*** Long queries, difficult queries, query type

How to combine all these easily and appropriately?

5

5

Terrier Team | University of Glasgow

FEATURES FOR LEARNING

6

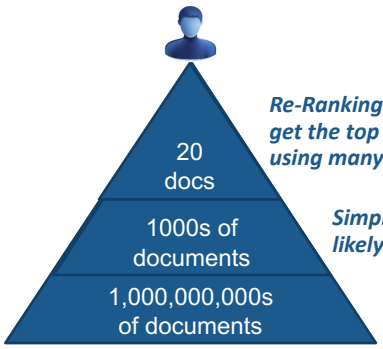
6

Ranking Cascades

Terrier Team | University of Glasgow

Typically, in web-scale search, the ranking process can be seen as a series of cascades [1]

- Rank some documents
- Pass top-ranked onto next cascade for refined re-ranking



Re-Ranking: Try really hard to get the top of the ranking correct, using many signals (features) ⇨ *LEARNING TO RANK*

Simple Ranking: Identify a set most likely to contain relevant documents ⇨ *e.g. BM25*

Boolean: Do query terms occur? ⇨ *e.g. AND/OR*

[1] J Pederson. Query understanding at Bing. SIGIR 2010 Industry Day.

7

7

Learning to Rank

Terrier Team | University of Glasgow

Application of tailored machine learning techniques to automatically (select and) weight retrieval *features*

- Based on **training data** with relevance assessments

Learning to rank has been popularised by commercial search engines (e.g. Bing, Baidu, Yandex)

- They require large training datasets, possibly instantiated from click-through data
- **Click-through data** has facilitated the deployment of learning approaches

T.-Y. Liu. (2009). Learning to rank for information retrieval. Foundation and Trends in Information Retrieval, 3(3), 225–331.

8

8

Types of Features

Terrier Team | University of Glasgow

Typically, commercial search engines use hundreds of features for ranking documents, usually categorised as follows:

Name	Varies depending on…		Examples
	Query	Document	
Query Dependent Features	✓	✓	Weighting models, e.g. BM25, PL2 Proximity models, e.g. Markov Random Fields Field-based weighting models, e.g. PL2F Deep-learned semantic matching
Query Independent Features	✗	✓	PageRank, number of inlinks Spamminess
Query Features	✓	✗	Query length Presence of entities

NB: Unlike text classification, features in LTR do NOT include the presence of a *specific* word in the document

9

9

Learning to Rank

Terrier Team | University of Glasgow

1. *Candidate* Identification

- Apply BM25 or similar (e.g. DFR PL2) to rank documents with respect to the query (list of candidate documents)
- Hope that the candidate ranking contains *enough* relevant documents

2. Compute more *features*

- Query Dependent
- Query Independent
- Query Features

3A. Learn ranking model

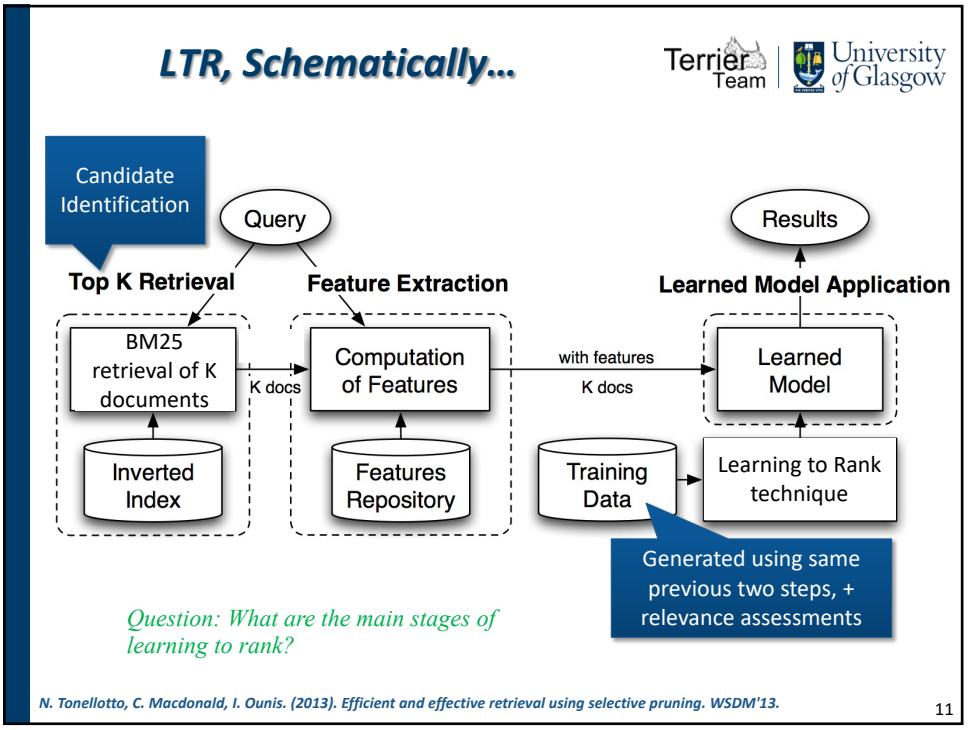
- Based on training data

3B. Apply learned model

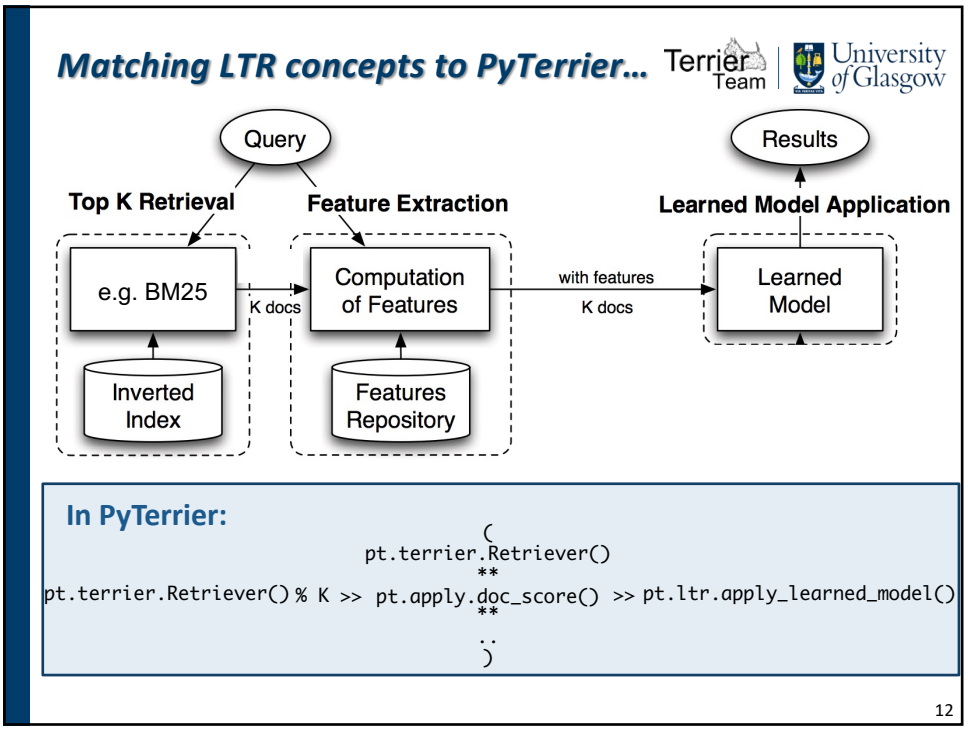
- Re-rank candidate documents

10

10


LTR, Schematically…
Terrier Team
University of Glasgow
Candidate Identification
Query
Top K Retrieval
Feature Extraction
Results
Learned Model Application
BM25 retrieval of K documents
K docs
Computation of Features
with features
K docs
Learned Model
Inverted Index
Features Repository
Training Data
Learning to Rank technique
Generated using same previous two steps, + relevance assessments
Question: What are the main stages of learning to rank?
N. Tonellotto, C. Macdonald, I. Ounis. (2013). Efficient and effective retrieval using selective pruning. WSDM'13.
11

11


Matching LTR concepts to PyTerrier…
Terrier Team
University of Glasgow
Query
Top K Retrieval
Feature Extraction
Results
Learned Model Application
e.g. BM25
K docs
Computation of Features
with features
K docs
Learned Model
Inverted Index
Features Repository
In PyTerrier:
(
pt.terrier.Retriever()
**
pt.terrier.Retriever() % K >> pt.apply.doc_score() >> pt.ltr.apply_learned_model()
**
..
)
12

12

"Featured" Data Model

Terrier Team | University of Glasgow

We've seen the R dataframe type in PyTerrier

qid	query	docno	score	rank
q1	chemicals	d5	14.9	0

Retrieved documents with features, R_f:

qid	query	docno	score	rank	features
q1	chemicals	d5	14.9	0	[1, 3.6, 9, 0.4]

- The features column contains, for each document, an **array** of additional feature scores.
- These can be used for learning and applying LTR models
- Applying an LTR model would override the original score of the document, causing a re-ranking

13

13

Feature Union Operator

Terrier Team | University of Glasgow

Feature Union (**)

- Say we want to take multiple retrieval approaches as different features. We use T1 ** T2 to denote, that given a set of input documents, use these transformers to obtain additional *feature* scores
- Formally:

$$Rf = (T_1 \ ** \ T_2)(R) :=$$
$$R_1 = T_1(R), \quad R_2 = T_2(R)$$
$$Rf = (R_1 \bowtie R_2)[[f_1, f_2] \rightarrow f]$$

- E.g.

```
Retriever("BM25") >>      ( Retriever("Tf") ** Retriever("PL2") )
```

R

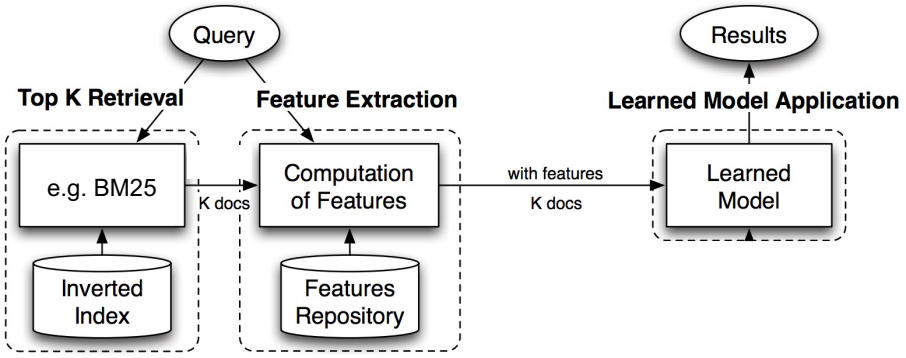
qid	docno	score
1 q1	d5	(bm25 score)

R_f

qid	docno	score	features
1 q1	d5	(bm25 score)	[tf score, pl2 score]

14

Matching LTR concepts to PyTerrier…



In PyTerrier:

```
pt.terrier.Retriever() % K >> pt.apply.doc_score() >> pt.ltr.apply_learned_model()
```

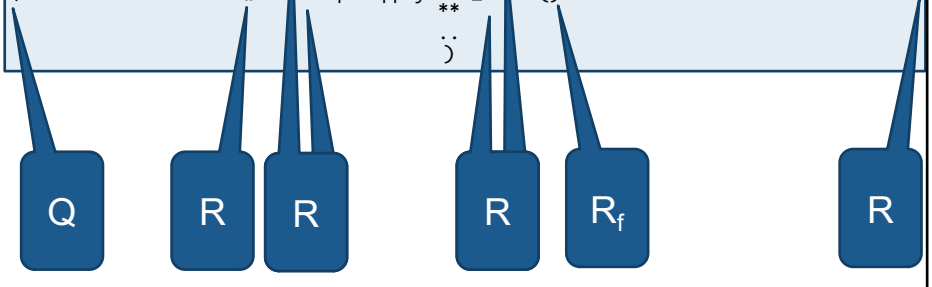
15

15

Data Model Transformations

In PyTerrier:

```
pt.terrier.Retriever() % K >> pt.apply.doc_score() >> pt.ltr.apply_learned_model()
```



16

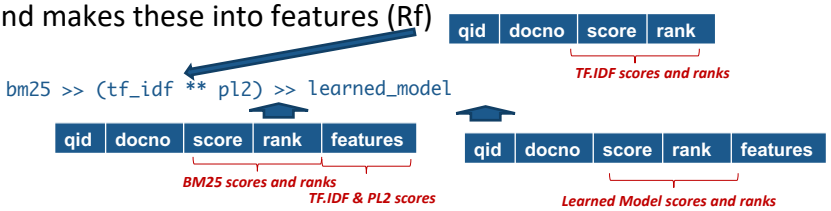
16

Summary: LTR Support in PyTerrier

Terrier Team | University of Glasgow

PyTerrier has excellent support for Learning to Rank:

- >> "then" operator allows a multi-stage retrieval pipeline (candidates, feature extraction, learned model reranking)
- ** "feature-union" operator combines multiple rerankers (R→R) and makes these into features (Rf)



- pt.apply transformers for calculating 1 or >1 feature per doc.
- Support for common training and applying learning-to-rank techniques (from SKLearn to gradient boosted trees)

NB: Exercise 2 has a standardised feature list, to which you will add new simple URL length and proximity features

17

17

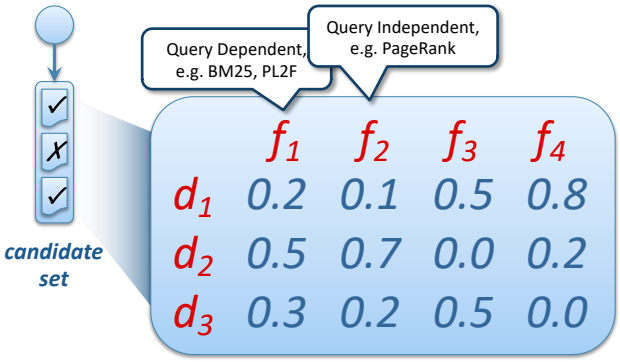
Features

Terrier Team | University of Glasgow

Query dependent
- Weighting models
- … including fields
- Proximity/ngram

Query independent
- Link Analysis, e.g. PageRank Score
- URL length
- Spam Score
- Document Type (PDF, Wikipedia)

Query features
- Query length
- Presence of entities
- Predicted query performance



Question: What are the three types of features in learning to rank?

18

18

Query-Dependent Feature Extraction

Terrier Team | University of Glasgow

We might deploy several query dependent features

- Such as additional weighting models, e.g. fields/proximity, that are calculated based on information in the inverted index
- e.g. `bm25 >> (tf_idf ** pl2) >> LTR`

We want the result set passed to the final cascade to have all query dependent features computed

- Once first cascade retrieval has ended, it's **too late** to compute features without **re-traversing** the inverted index postings lists
- It would take **too long** to compute all features for all scored documents

Solution: cache the postings for documents that *might* make the candidate ranking

- Postings contain frequencies, positions, fields, etc.

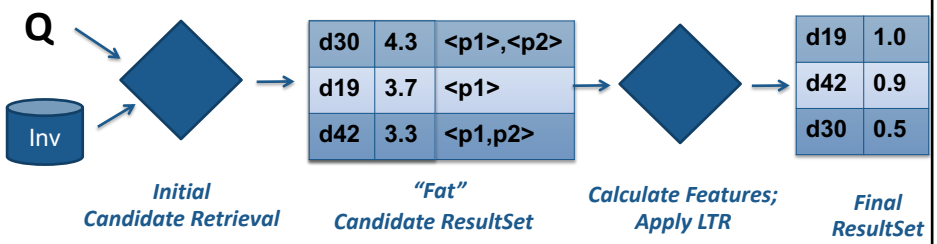
20

20

Query-Dependent Feature Extraction

Terrier Team | University of Glasgow

Solution: cache the postings for documents that *might* make the candidate set



In PyTerrier this is exposed via pt.terrier.FeatureRetriever… a single Transformer that does first-pass retrieval and calculates more query dependent features

C Macdonald et al. (2012). About Learning Models with Multiple Query Dependent Features. TOIS 31(3).

21

21

Query-Dependent Feature Extraction: Pipeline Equivalence in PyTerrier…

Terrier Team | University of Glasgow

Using this "fat" postings via FeaturesRetriever

```
pipe_slow = pt.terrier.Retriever(wmodel='BM25')
   >> (
          pt.terrier.Retriever(wmodel='TF_IDF') **
          pt.terrier.Retriever(wmodel='PL2')
   ) >> LTR
```

3 inverted file accesses for each query term

can be replaced with

```
pipe_fast = pt.terrier.FeaturesRetriever(
          wmodel='BM25',
          features=['WMODEL:TF_IDF', 'WMODEL:PL2'])
    >> LTR
```

1 inverted file access for each query term

NB: We provide the starting featureset in Ex2 – you need only worry about adding new URL length and proximity features using the ** operator

22

22

Terrier Team | University of Glasgow

Types and Examples of Learning to Rank Algorithms

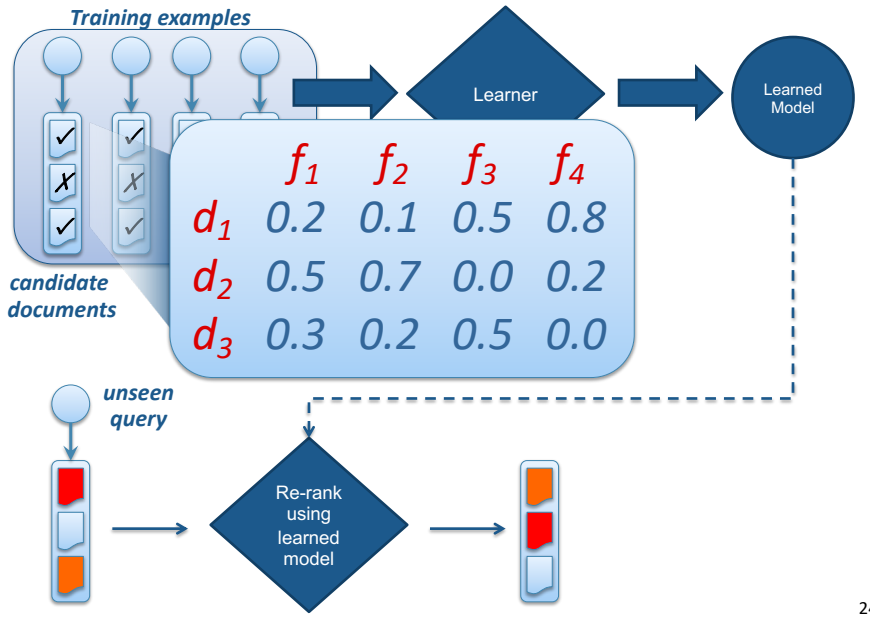
LEARNING PROCESS

23

23

Learning to Rank Process

Terrier Team | University of Glasgow



24

24

Training & Evaluating Learning-to-Rank

Terrier Team | University of Glasgow

We need lots of queries to learn an effective, robust model

We cannot measure the success of a learning-to-rank technique on one of the queries it has been trained upon

- Such training performance is likely to **overestimate** its effectiveness compared to *unseen* queries

Instead, we need to separate the queries in our test collection into disjoint sets: for training the model; for *tuning* the learning hyperparameters (the validation query set); and for testing how effective the model is for unseen queries

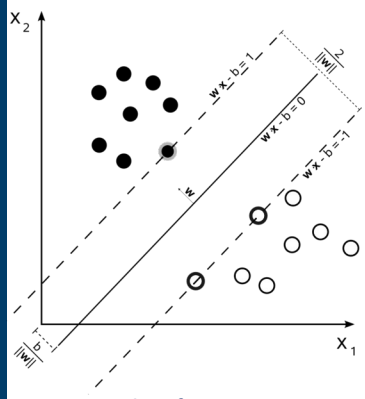
- We use the same processing (candidate retrieval and features) for queries in each set
- (More on validation shortly)

25

25

Types of Learning

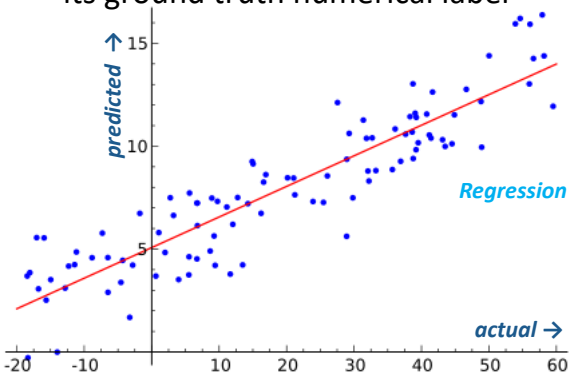
Terrier Team | University of Glasgow



Classification

Ranking is a form of supervised machine learning. Slightly different from classification, more closely related to regression:

- For each instance, aims to predict its ground truth numerical label



Regression

26

Types of Learning for Ranking Tasks (1)

Terrier Team | University of Glasgow

A 'loss function' is used to estimate how good a particular model is (based on the training data)

- The learner will try many different models (to find the best one)

There are three types of learning appropriate for ranking tasks: pointwise, pairwise & listwise

- Each differ in the way that their loss function is computed

Pointwise approaches are closest to classification/regression

- They aim to predict the correct relevance label
  - ***Classification***: how many documents did the learner correctly predict as relevant/non-relevant?
  - ***Regression***: how different is the predicted score from the correct relevance label for each document

OPRF: N Fuhr. (1989). Optimal Polynomial Retrieval Functions Based on the Probability Ranking Principle. TOIS. 7(3), 1989.
GBRT: S. Tyree, K.Q. Weinberger, K. Agrawal, J. Paykin: Parallel boosted regression trees for web search ranking. WWW 2011.

27

27

Types of Learning for Ranking Tasks (2)

Terrier Team | University of Glasgow

Pointwise techniques suffer from a limitation for ranking tasks, in that they consider each document independently, instead of trying to get the relevant documents ranked ABOVE the non-relevant ones

Pairwise

- Take two documents with some preference
  - E.g. different relevance labels, 0 and 1, less and more clicks
- Try to minimise a loss function based on the ranking error between pairs of documents
- Problem? Doesn't always approximate a real evaluation measure
- Examples: RankSVM, RankNet

Listwise

- Consider the entire ranking of documents
- Encapsulates a standard IR evaluation measure within the loss function
- Not all measures are appropriate for list wise learning, e.g. P@3 vs NDCG@1000
- Examples: **AFS, LambdaMART**
- Generally more effective than Pointwise or Pairwise

RankSVM: T. Joachims. Optimizing Search Engines using Clickthrough Data. CIKM'03.
RankNet: C Burges et al. Learning to Rank using Gradient Descent. ICML'05.
AFS: D. Metzler. Automatic Feature Selection for the Markov Random Field Model for IR. CIKM'07.
LambdaMART: C. J. Burges. From RankNet to LambdaRank to LambdaMART: An Overview. Technical Report MSR-TR-2010-82, Microsoft Research.

28

28

Terrier Team | University of Glasgow

LINEAR MODELS FOR LTR

29

29

Types of Learned Models (1) Linear Model

Terrier Team | University of Glasgow

Linear Models (the most intuitive to comprehend)

- Many learning to rank techniques generate a linear combination of feature values:

$$score(d,Q) = \sum_f w_f \cdot value_f(d)$$

- Linear Models make some assumptions:
  - ***Feature Usage***: They assume that the same features are needed by all queries
  - ***Model Form***: The model is only a linear combination of feature values.
    - Contrast this with genetic algorithms, which can learn functional combinations of features, by randomly introducing operators (e.g. try divide feature *a* by feature *b*), but are unpractical to learn
- **Question**: How do we set w_f values that maximise the performance of an IR evaluation metric?

30

30

Difficulty in Training

Terrier Team | University of Glasgow

IR evaluation measures are not continuous, and hence are not differentiable wrt a feature weight

- i.e. they are not smooth as we vary a feature weight *w*

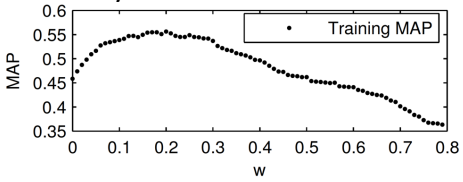
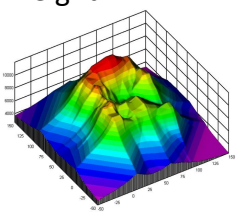


Figure 2: Tuning w, for *log* PageRank combination.

Why?



Metrics don't respond to changes in the scores of documents, but only when a swap between the ordering of two documents takes place

- We need advanced gradient descent/hill-climbing/simulated annealing
- Many of the advances in learning to rank try to address this

Q: Why are evaluation measures non-smooth wrt features weights?

Figures from: (a) Craswell et al., SIGIR 2005 (b) Hongning Wang

31

31

(Greedy) Automatic Feature Selection (AFS)

Terrier Team | University of Glasgow

Consider a linear model:

$$score(d,Q) = \sum_f w_f \cdot value_f(d)$$

AFS: Select a $w_f > 0$ for the feature f that best improves an evaluation measure M

	f_1	f_2	f_3	f_4
d_1	0.2	0.1	0.5	0.8
d_2	0.5	0.7	0.0	0.2
d_3	0.3	0.2	0.5	0.0

Step 1: Select the most effective single feature according to M

f1	0.30
f2	0.35
f3	0.05
f4	0.10

→ *(f2, 1)*

w_1	w_2	w_3	w_4

Current Best M = 0

D. Metzler. Automatic Feature Selection for the Markov Random Field Model for IR. CIKM'07.

32

32

(Greedy) Automatic Feature Selection (AFS)

Terrier Team | University of Glasgow

Consider a linear model:

$$score(d,Q) = \sum_f w_f \cdot value_f(d)$$

AFS: Select a $w_f > 0$ for the feature f that best improves an evaluation measure M

Step 2: Select the feature and corresponding w_f that most improves M (e.g. using sim. annealing)

	f_1	f_2	f_3	f_4
d_1	0.2	0.1	0.5	0.8
d_2	0.5	0.7	0.0	0.2
d_3	0.3	0.2	0.5	0.0

f2 + ?	x f1		
f2 + ?	x f3		
f2 + ?	x f4		

w_1	w_2	w_3	w_4
	1		

Current Best M = 0.35

D. Metzler. Automatic Feature Selection for the Markov Random Field Model for IR. CIKM'07.

33

33

(Greedy) Automatic Feature Selection (AFS)

Terrier Team | University of Glasgow

Consider a linear model:

$$score(d,Q) = \sum_f w_f \cdot value_f(d)$$

AFS: Select a $w_f > 0$ for the feature f that best improves an evaluation measure M

Step 2: Select the feature and corresponding w_f that most improves M (e.g. using sim. annealing)

	f_1	f_2	f_3	f_4
d_1	0.2	0.1	0.5	0.8
d_2	0.5	0.7	0.0	0.2
d_3	0.3	0.2	0.5	0.0

f2 + 1.3 x f1	0.36
f2 + 0.3 x f3	0.40
f2 + 0.9 x f4	0.34

→ *(f3, 0.3)*

w_1	w_2	w_3	w_4
	1	0.3	

Current Best M = 0.40

D. Metzler. Automatic Feature Selection for the Markov Random Field Model for IR. CIKM'07.

34

34

(Greedy) Automatic Feature Selection (AFS)

Terrier Team | University of Glasgow

Consider a linear model:

$$score(d,Q) = \sum_f w_f \cdot value_f(d)$$

AFS: Select a $w_f > 0$ for the feature f that best improves an evaluation measure M

Step 2: Select the feature and corresponding w_f that most improves M (e.g. using sim. annealing)

	f_1	f_2	f_3	f_4
d_1	0.2	0.1	0.5	0.8
d_2	0.5	0.7	0.0	0.2
d_3	0.3	0.2	0.5	0.0

f2 + 1.3 x f1	0.36
f2 + 0.3 x f3	0.40
f2 + 0.9 x f4	0.34

→ *(f3, 0.3)*

Step 3: Goto step 2, until we observe no more improvements in M, or no more features

w_1	w_2	w_3	w_4
	1	0.3	

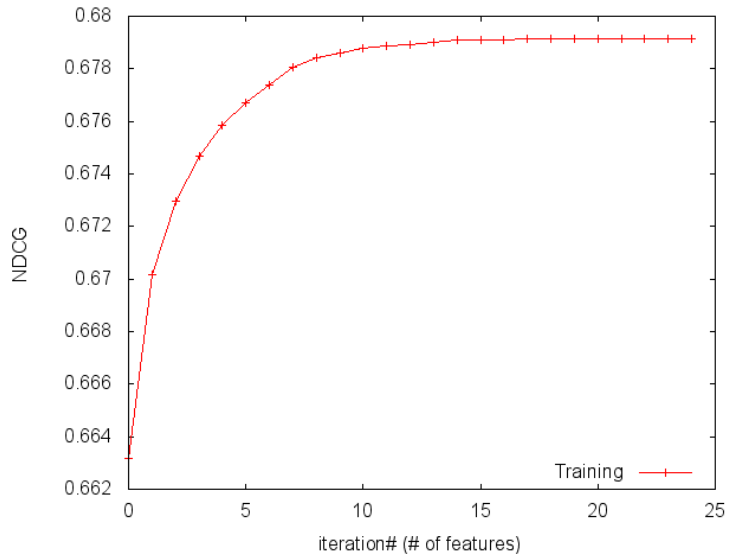
Current Best M = 0.40

D. Metzler. Automatic Feature Selection for the Markov Random Field Model for IR. CIKM'07.

35

35

Training performance, by Iteration

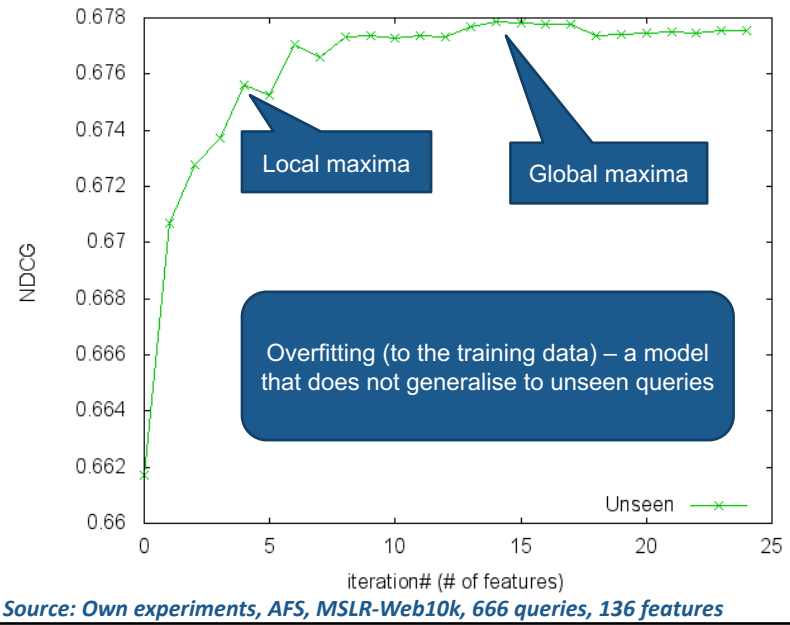


Source: Own experiments, AFS, MSLR-Web10k, 1334 queries, 136 features

36

36

Performance on Unseen Data

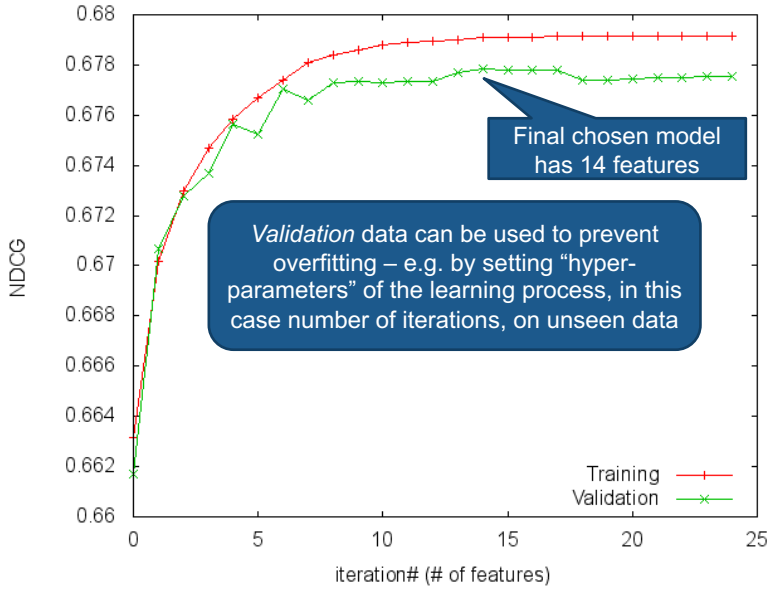


Source: Own experiments, AFS, MSLR-Web10k, 666 queries, 136 features

37

37

Solution: Use Validation Data



38

38

Summary: Evaluating a Learning to Rank Technique

Evaluation of learning to rank (L2R) uses a test collection

- Documents & queries with known relevant documents

We split queries into three subsets:

- Training – for learning the model
- Validation – for setting hyper-parameters, e.g. number of iterations. Unseen to the learner; prevents overfitting to the training data
- Testing – for determining how effective the learner is on unseen queries

In Exercise 2, you will evaluate on the HP04 (test) topics;
We have provided separate training and validation topic sets.
c.f. PyTerrier's L2R documentation on how to learn using training & validation topics

39

39

Terrier Team | University of Glasgow

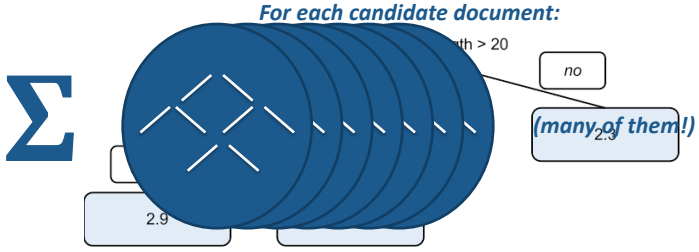
REGRESSION TREES FOR LTR

Type of Learned Models (2) Regression Trees

Terrier Team | University of Glasgow

Tree Models

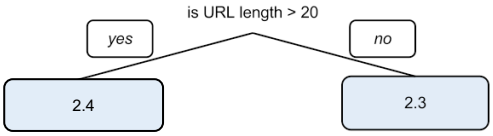
- A *regression tree* is series of decisions, leading to a partial score output
- The outcome of the learner is a "forest" of many such trees, used to calculate the final score of a document for a query
- Their ability to customise branches makes them more effective than linear models
- Regression trees are **pointwise** in nature, but several major search engines have created adapted regression tree techniques that are listwise
  - E.g. Microsoft's LambdaMART is at the heart of the Bing search engine



S. Tyree, K. Weinberger, K. Agrawal, J. Paykin. (2011). Parallel Boosted Regression Trees for Web Search Ranking. WWW'11.

Growing a Tree

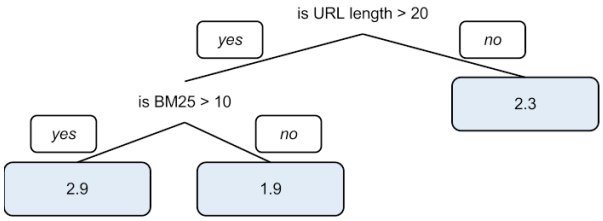
Terrier Team | University of Glasgow



	f_1	f_2	f_3	f_4	L
d_1	0.2	0.1	0.5	0.8	1
d_2	0.5	0.7	0.0	0.2	2
d_3	0.3	0.2	0.5	0.0	0

Loss = 1.4

Q: Can we add another decision point?



Loss = 1.2 ⟹ *Gain = 0.2*

42

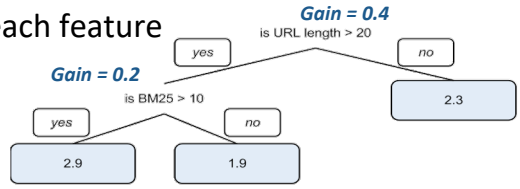
42

Feature Importance

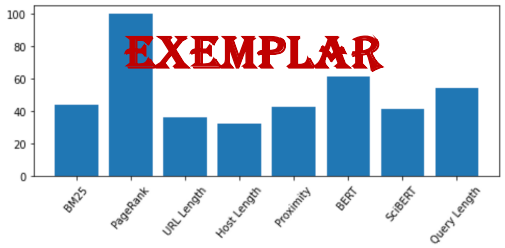
Terrier Team | University of Glasgow

Regression Trees have an in-built measure of feature importance

- By recording the "gain" at each node during training time
- Summing the gains for each feature



Feature	Norm. Total Gain
BM25	45
PageRank	100
URL Length	39
…	…



NB: sklearn's RandomForest, as well as XGBoost and LightGBM, all provide a `feature_importances_` variable

43

43

Summary of LTR Types

Terrier Team | University of Glasgow

Some "take-aways":

- Listwise techniques are typically more effective; linear models are easy to understand; regression trees are very adaptable
  - LambdaMART enhances pointwise regression trees with a listwise measure
  - Currently one of the benchmarks to beat!
- Validation data is equally important to use for all types of supervised learning, including LTR, to prevent overfitting
  - Always use a validation set!

In Exercise 2 of your coursework, you will be experimenting with the state-of-the-art LambdaMART technique, as provided by LightGBM

44

44

Example Performances

Terrier Team | University of Glasgow

Example performances on 8000 queries from a Microsoft learning to rank dataset

Learning to Rank Technique	Type	NDCG@10	+	-
BM25	n/a	0.2987	-	
AFS (Simulated Annealing)	Listwise: Linear	0.4263	5596	1778
Boosted Regression Trees	Pointwise: Tree	0.4240	5802	1808
LambdaMART	Listwise: Tree	**0.4373**	5760	1844

- Differences might be small in magnitude, but are statistically significant over hundreds of queries

In Exercise 2 of your coursework, you will be experimenting with the stateof-the-art LambdaMART technique, as provided by LightGBM

45

45

The Importance of Candidate Ranking Size

Terrier Team | University of Glasgow

How many candidates is necessary for effective learning?

- Small nbr of candidates: faster learning, faster retrieval
- Large nbr of candidates : higher recall

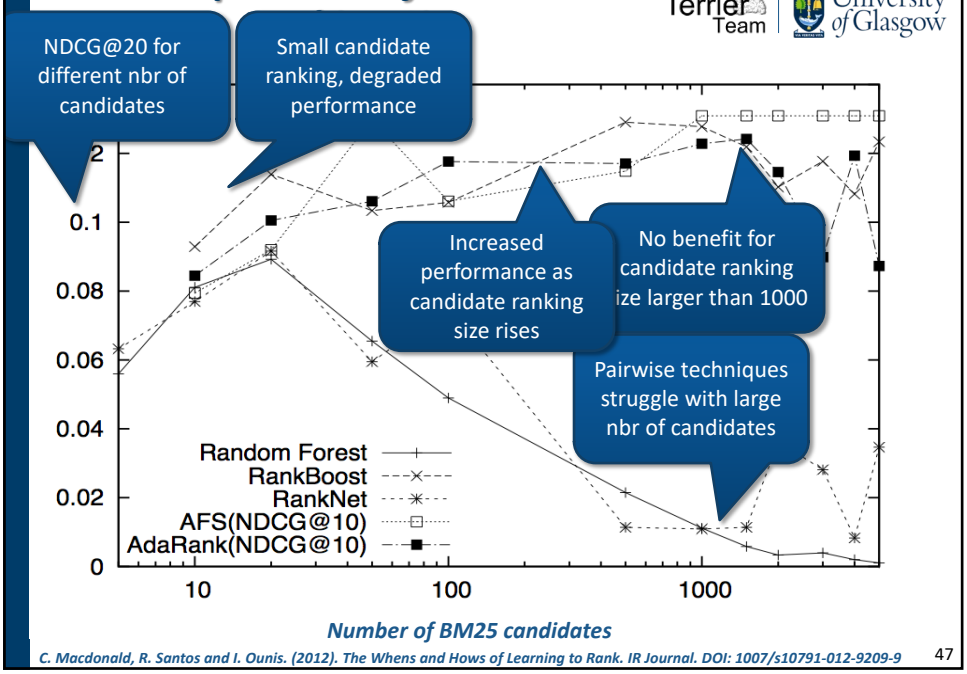
(See next figure)

C. Macdonald, R. Santos and I. Ounis. (2012). The Whens and Hows of Learning to Rank. IR Journal. DOI: 1007/s10791-012-9209-9 46

46

The Importance of Candidate

Terrier Team | University of Glasgow



C. Macdonald, R. Santos and I. Ounis. (2012). The Whens and Hows of Learning to Rank. IR Journal. DOI: 1007/s10791-012-9209-9 47

47

Learning to Rank Best Practices (Basics)

Terrier Team | University of Glasgow

Q: How many candidate documents should be reranked in learning-to-rank?

About the candidate ranking – it must be:

- **Small enough** that calculating features doesn't take too long
- **Large enough** to have enough sufficient recall of relevant documents
  - Previous slide says 1000 documents needed for TREC Web corpora
- **Simple early cascade**: Only time for a single weighting model to identify the candidate documents
  - E.g. PL2 < BM25 with no features; but when features are added, no real difference

Multiple weighting models as features:

- Using more weighting models do improve the learned model

Field Models as Features

- Adding field-based models as features always improves effectiveness

In general, Query Independent features (e.g. PageRank, Inlinks, URL length) improve effectiveness

C. Macdonald, R. Santos, I. Ounis, B. He. About Learning Models with Multiple Query Dependent Features. TOIS 31(3), 2013.
C Macdonald & I Ounis (2012). The Whens & Hows of Learning to Rank. INRT. 16(5), 2012.

48

48

Learning to Rank Best Practices (Query Features)

Terrier Team | University of Glasgow

Tree-based learning to rank techniques have an inherent advantage

- They can activate different parts of their learned model depending on feature values
- We can add "query features", which allow different sub-trees for queries with different characteristics
  - E.g. long query, apply proximity

Class of Query Features	NDCG@20
Baseline (47 document features) LambdaMART	*0.2832*
Query Performance Predictors	0.3033*
Query Topic Classification (entities)	**0.3109***
Query Concept Identification (concepts)	0.3049*
Query Log Mining (query frequency)	0.3085*

C. Macdonald, R. Santos and I. Ounis. (2012). On the Usefulness of Query Features for Learning to Rank. CIKM'12.

49

49

Summary

Terrier Team | University of Glasgow

Generative models such as LM and DFR are increasingly replaced by discriminative models

- However, such generative models have a role as strong features, or for candidate identification
- Web search nowadays relies on discriminative learning-to-rank techniques to perform fast, effective rankings (see next lectures)

Generally speaking, listwise approaches are the most effective family of learner type

- Techniques based on regression trees (e.g. Boosted Regression Trees, LambdaMART) are considered to be the state-of-the-art [at least in Web search!]

There are increasingly more tools to deploy such algorithms

- LightGBM, Xgboost, Ranklib, TF-Ranking

To some extent being replaced by Neural IR…

50

50

Coming Soon: Neural Ranking Models

Terrier Team | University of Glasgow

LTR uses *hand-engineered* features (BM25, PL2F, PageRank etc)

- More research is now focused on learning the ranking model from bare text. This is achievable because of advances in **neural network architectures**

1. Word2vec: shallow neural model that creates a dense representation of words.

- Each word can be represented by an *embedding* vector of say numbers; similar words have similar vectors
- Trained to predict a word given its *context* words

2. BERT: deep attention-based contextualised embeddings invented by Google aka Neural Network Language Models (NNLM)

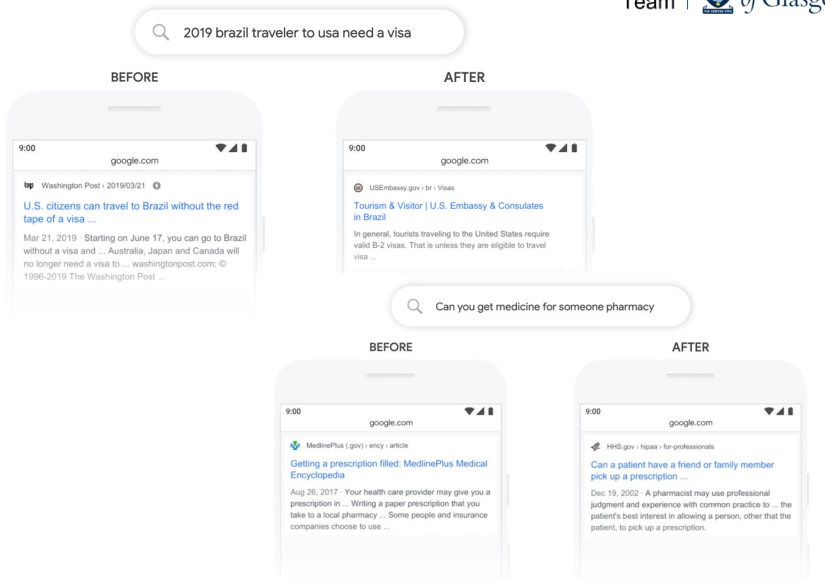
- Uses attention while training the neural network to predict the *next sentence*. Attention helps identify which contextual words affect the meaning of a given word
- Difficult to train, but can be *fine-tuned* to adapt to a new task
- Has proven to be **very effective** at ranking, but **slow** (400 paragraphs/sec on a "commodity GPU" card

51

51

BERT in Action

Terrier Team | University of Glasgow



Google reports that BERT is quite effective for long queries

52

52

Contrast: LTR vs NNLM

Terrier Team | University of Glasgow

LTR dataset:

0 qid:1 1:3 2:0 3:2 4:2 … 135:0 136:0
2 qid:1 1:3 2:3 3:0 4:0 … 135:0 136:0

Relevance Label | Query ID | Hand-Engineer Features Values (QD/QI)

NNLM learning dataset

1 188714 1000052 foods and supplements to lower blood sugar Watch portion sizes: Even healthy foods will cause high blood sugar if you eat too Make sure each of your meals has the

Relevance Label | Query ID | Query Text | Document text

Neural ranking learns representation for queries and documents, and how to measure their similarity, directly from the labels

- Needs MUCH more data than LTR – e.g. MSMARCO 300k training queries

Neural models can function as *re-rankers,* or end-to-end with embedded indexing and query representations (*dense* retrieval)

More on Neural IR in Lecture 10

53

53

Final thoughts/Recent Research: Risk-sensitive LTR

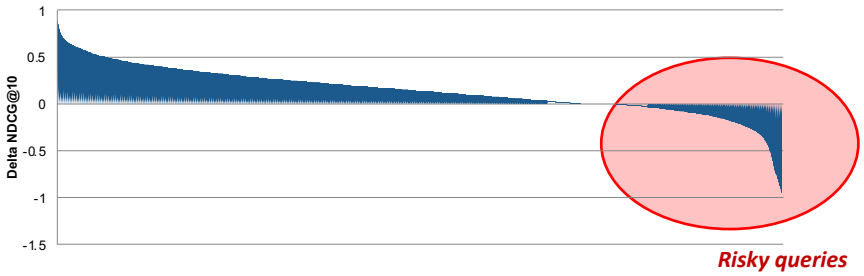
Terrier Team | University of Glasgow

Learning to rank improves most queries over the candidate generation model (e.g. BM25), but not all

- Risk occurs when a LTR system underperforms compared to the candidate identification model for a given query

BM25: Mean NDCG@10 = 0.2987
L'MART: Mean NDCG@10 = 0.4373

Improvement in applying LambdaMART (n=8000 queries)



Risky queries (1844)

54

54

Avoiding Risk

Terrier Team | University of Glasgow

Why is risk bad?

Search engines want to satisfy users for every single query issued to the system

- Unsatisfied users might switch engine, leading to loss in trust and consequently ad revenue, etc

Hence, a search engine aims to avoid any abject failures - i.e. queries where performances decrease viz. of a minimum standard:

- Learning to rank should not hurt performance, e.g. > BM25, PL2, etc

55

55

Risk-sensitive LTR

Terrier Team | University of Glasgow

We have been working on this problem: how can we make LambdaMART more risk-averse during learning

- i.e. create models that identify poor performing queries and learn to resort to more stable models (e.g. BM25)

We do this by emphasising reductions in effectiveness compared to BM25 during learning

- So trees that are risky are not learned

Let *ΔM* be the change in a metric for query *j* for a given tree

$$\Delta M' = \begin{cases} \Delta M & \text{if } M_m(j) + \Delta M \geq M_b(j); \\ \Delta M \cdot (1+\alpha) & \text{otherwise.} \end{cases}$$

- So, here we emphasise *ΔM* by *α* if the tree is poor (*α*>1)

Let us know if this kind of research sounds interesting!

B.T. Dinçer, C Macdonald and I Ounis. Hypothesis Testing for Risk-Sensitive Evaluation of Retrieval Systems. SIGIR '14.

56

56

Acknowledgements

Terrier Team | University of Glasgow

A couple of slides are based on:

- Bruce Croft et al. – see book: "Search Engines: Information Retrieval in Practice"
- Hongning Wang's presentation

The following people gave input to the creation of these slides:

- Iadh Ounis
- Richard McCreadie

57

57